

OTP-Hunter: An App-Based Fuzzing Framework to Discover One Time Password Vulnerabilities

Futai Zou , Senior Member, IEEE, Zehui Zhang , and Yuzong Hu

Abstract—One Time Password (OTP) plays a crucial role in verifying user identities for tasks like registration, login, and forgotten password on cloud platforms. However poorly implemented OTP can lead to vulnerabilities like Bomb attacks (a Denial of Service attack) on SMS or Email, OTP Brute-Force attacks, and OTP bypass, compromising access control. While fuzzing at the application layer (e.g., http) is effective for web products, it faces limitations in mobile applications due to cryptographic complexities and certificate verification. To address this, we propose a fuzzing framework, OTP-Hunter, combining static and dynamic analysis to uncover OTP vulnerabilities in applications with minimal manual intervention. We use OTP-Hunter to analyze 59 IoT-related apps (not limited), identify 68 vulnerabilities and reported them to the vendor and China National Vulnerability Database for 11 CNVDs. These vulnerabilities could let attackers remotely take control of devices by knowing the victim's cloud account, typically a phone number. Our results also show that compared with mutating the network requests, our approach can cover more circumstances like custom protocol because we do not need to know the encryption process.

Index Terms—Fuzzing, applications, one time password (OTP), vulnerability detection.

I. INTRODUCTION

ONE Time Password (OTP) is a dynamic password for user authentication. It is widely used in register, login and password reset where account ownership needs to be confirmed. OTPs are commonly delivered through Short Message Service (SMS) and Email. Flawed implementation of OTPs will lead to SMS Bomb and Email Bomb vulnerabilities. An SMS Bomb occurs when an attacker exploits weaknesses in the OTP delivery system, sending an overwhelming number of OTP messages to flood the victim's phone with spam. Similarly, an Email Bomb involves sending a massive volume of OTP emails to overwhelm the victim's email inbox. These attacks not only disrupt the normal functioning of the communication channels but can also compromise the security and usability of OTPs by making it difficult for the legitimate user to locate and use the valid OTP amidst the flood of spam messages. Email bomb attacks can be rented for 5 USD on darknet [1] and there are also many open-source SMS bomb tools on the internet [2], [3], [4]. Most

of these tools collect flawed OTP api in various websites to realize Denial of Service (DoS) attacks. Since these spams come from legitimate server (e.g., Tencent, NetEase), it is difficult for anti-spam tools to identify.

SMS and email bomb vulnerability may unconsciously make manufacturers' legitimate api become weapons for darknet attackers, which will cause reputation and financial loss to the company. Besides, OTP Brute-Force and OTP Bypass vulnerabilities will invalidate user authentication, resulting in leakage of user privacy data. In the Internet of Things (IoT) scenario, these vulnerabilities allow attackers to remotely take control of others' devices by simply knowing their cloud account, usually a phone number. This enables unauthorized access to and control over others' smart homes, posing a significant threat to user privacy. Therefore, discovering OTP vulnerabilities is valuable and bug bounty platforms define these kind of bugs as medium-severity vulnerabilities [5], [6], [7].

When we test OTP vulnerabilities on the web, the usual practice is to mutate and replay the network request packets. However when it comes to the apps, we face many **new challenges**. Firstly, many apps verify cloud certificates, preventing hackers from executing man-in-the-middle attacks to intercept network requests. Additionally, apps often employ digital signatures or encrypted key parameters to safeguard requests from tampering. Therefore, a hacker attempting to modify parameters at the network layer must engage in reverse engineering to comprehend the app's encryption algorithm.

To address challenges and identify OTP vulnerabilities in applications, we developed OTP-Hunter, a fuzzing framework designed for minimal manual intervention in testing OTP-related flaws. Our approach involves identifying cloud-related communication functions and mutating them to pinpoint OTP vulnerabilities. DIANE [8] is a tool designed for fuzzing IoT devices by identifying *fuzzing triggers*. Drawing from DIANE's methodology for identifying *fuzzing triggers*, we enhanced the static analysis and dynamic hook process to locate OTP-related functions. In DIANE's work *fuzzing triggers* are the functions executing before any data-transforming functions (e.g., network serialization), but after the input validation code. In our work, we integrate static analysis and dynamic hook techniques to pinpoint optimal hook functions (bypassing checks and identifying effective mutation points). Starting with network functions (such as com.android.volley, org.apache.http) and RESTful interfaces (such as Retrofit2 [9]), we construct a function call graph, trace their control flow, and extract all function chains that can reach these interfaces. Subsequently, we hook these function

Received 23 January 2024; revised 4 June 2025; accepted 13 July 2025. Date of publication 29 July 2025; date of current version 4 November 2025. This work was supported by the National Key Research and Development Program of China under Grant 2020YFB1807500. (Corresponding author: Futai Zou.)

The authors are with the School of Computer Science, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: zoufutai@sjtu.edu.cn; 2019zhangzehui@sjtu.edu.cn; huyz97@sjtu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2025.3593598

chains, focusing on functions containing parameters like email, telephone, or OTP codes. We then extract the function closest to the final sending interface with unencrypted parameters, which we term *cloud triggers* and use them as mutation points.

Unlike DIANE, we first shift the scenario to identifying OTP-related vulnerabilities, focusing on functions containing parameters such as email, phone numbers, or OTP codes. Second, we observe that DIANE fails to account for implicit invocations of asynchronous functions (e.g., RxJava2 and the Runnable interface), which leads to incomplete call graphs. Therefore, we specifically construct rules for these two interfaces to ensure complete extraction of call graphs. Finally, for network function identification, we also support widely used RESTful interfaces (e.g., the Retrofit2 framework). This adaptation enhances the tool's versatility and improves its success rate in detecting network functions.

Although our proposed method is fundamentally generic and can be applied to mobile applications for detecting OTP-related security vulnerabilities, we exclusively conducted experiments on IoT-related apps for two main reasons: (1) OTP Brute-Force and OTP bypass vulnerabilities allows attackers to remotely take control of others' devices with just knowledge of their cloud account (typically a phone number). This enables unauthorized access to and manipulation of other individuals' smart home devices, compromising the privacy of users. (2) IoT vendors design different apps for various usage scenarios (commercial, smart homes, etc.), but they use similar cloud APIs. This similarity in design leads to common flaws and a more widespread impact.

We have conducted in-depth research on 59 IoT-related apps and found 68 OTP-related vulnerabilities. Experiments also show that the OTP-Hunter can cover more test cases (e.g. adding special characters like '\x00', '\n') and save the time of manual reverse engineering, compared with fuzzing network requests and replaying UI operation.

Our main contributions are summarized as follows:

- We proposed a general fuzzing framework based on dynamic and static analysis for discovering OTP-related vulnerabilities on apps. This framework can automatically test four types of OTP vulnerabilities with minimal manual intervention. It eliminates the need for knowledge about encryption algorithms and communication protocols between apps and clouds.
- We enhanced our static analysis tool by incorporating specific rules tailored for widely used frameworks in Android applications. This modification enables better adaptation to popular RESTful frameworks and asynchronous function calls.
- We leveraged the characteristics of OTP vulnerabilities and dynamic hook techniques to identify the best fuzzing point, referred to as *cloud triggers*. This allows fuzzing to operate without being influenced by validation procedures, and the fuzzing messages can be accepted by the cloud server.
- We designed a prototype OTP-Hunter and evaluated it on 59 IoT-related apps. In total, it has found 68 OTP-related vulnerabilities from them. We have reported them to the vendors and obtained 11 CNVDs from China National Vulnerability Database.

Register

China >

Mobile Number/Email

Get Verification Code

☒ I Agree [User Agreement](#) and [Privacy Policy](#)

Fig. 1. Register UI of LenovoSmart App.

To foster future research, we will release the vulnerabilities detected by OTP-Hunter as well as the experiment data at <https://github.com/Netsec-SJTU/OTP-Hunter>

II. MOTIVATION

The design goal of OTP-Hunter is to build an automated fuzzing framework that discovers OTP-related vulnerabilities in IoT applications with minimal manual effort. OTP vulnerabilities are a type of business logic vulnerabilities that are caused by design flaws in cloud platforms. Due to this, we lack access to the vulnerability's source code. Additionally, monitoring poses a challenge as we cannot directly observe the operational status of the cloud platform. Instead, we infer it from the cloud platform's responses. Overall, discovering OTP-related vulnerabilities in an application aligns more closely with black-box fuzz testing.

In this section, we first introduce the scope of vulnerabilities we focus on. Then we use a real-world example to demonstrate the challenges that we must overcome.

A. Vulnerability Scope

In this paper, we focus on the vulnerabilities related to OTP. The use of OTP usually occurs in the login, registration, and password reset of the apps. For example, Fig. 1 is the registration interface of the LenovoSmart app [10], in which the area code, mobile phone number, and email address can be used as the fuzzing point. When we get OTP code to verify the ownership of the account, the procedure will be like Fig. 2.

Threat Model: Since the cloud account is usually the user's phone number, we assume that the attacker can use the victim's cloud account to initiate the cloud's OTP authentication process, as shown in Fig. 3. The cloud system cannot promptly confirm the requester's identity until step 6. The steps visible to the attacker during this process are 1, 4, 6, and 8. Due to design flaws in various steps, the logical vulnerabilities in cloud management can be categorized as follows:

- *OTP Leakage Vulnerability (Step 4):* If login verification is on the application side, the cloud will return the generated OTP value included in the response packet. This allows an attacker to directly obtain the OTP value to complete authentication and steal the victim's cloud account.

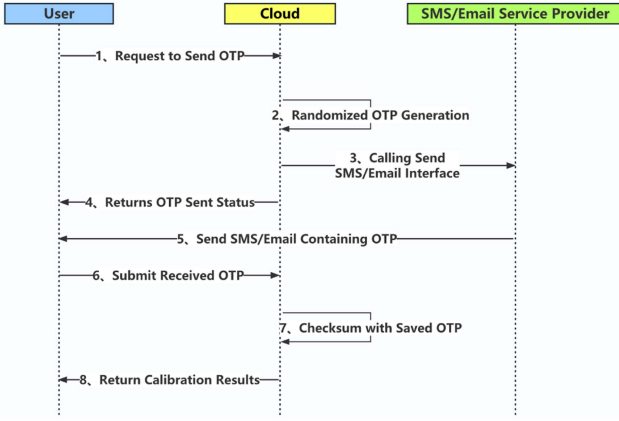


Fig. 2. Process of identity verification based on OTP code.

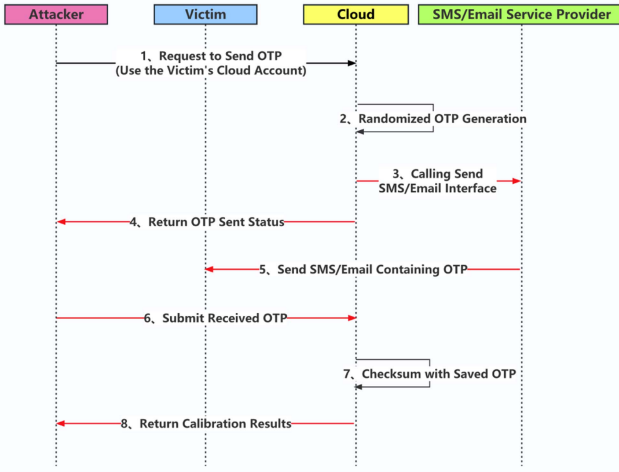


Fig. 3. Process of attacker pretending victim to request OTP authentication.

- **OTP Brute-Force Vulnerability (Step 6):** If the cloud does not limit the number of attempts for Step 6, the attacker can engage in brute force enumeration all combinations of 4-digit/6-digit authentication codes (SMS authentication codes usually contain only digits). This allows the attacker to discover the correct OTP value, completing the authentication and potentially stealing the victim's cloud account.
- **OTP Bomb Vulnerability (Step 5):** If the cloud does not limit the number of OTP requests for a single cloud account, an attacker can initiate the OTP identity verification process for a specified victim an unlimited number of times. This results in a large number of replays of Step 5, so that the victim's phone and mailbox receive a large number of OTP messages, which disrupts the victim's normal use.
- **Resource Consumption Vulnerability (Step 3):** If the cloud restricts requests for a single account, the attacker can continue initiating OTP authentication for different cloud accounts. This leads to the cloud consistently consuming SMS and email resources. Moreover, since API interfaces for operators or third-party services are typically billed

per use, this vulnerability results in a substantial waste of resources for vendors.

By analyzing the threat model, we can know that the cause of such logical vulnerabilities are due to the design flaws in the cloud. The attackers can use the flaws to destroy certain steps in the OTP authentication process, and then complete the authentication bypass and other attacks. Since the cloud account management process is related to OTP, cloud account management vulnerabilities are also referred to as OTP-related vulnerabilities below.

B. Challenges

The cause of OTP-related vulnerabilities lies in the design flaws of the cloud. Since we cannot access the source code of the cloud, we rely on the black-box testing method to mine vulnerabilities. Currently, there have been some research advancements in the exploration of OTP-related vulnerabilities.

Burp Suite [11] is a traditional web logic vulnerability exploration tool. The primary approach involves utilizing man-in-the-middle attacks to intercept HTTP or HTTPS request packets. Specific parameters (e.g., phone number field) are then mutated and replayed to observe responses, determining the presence of vulnerabilities. It has also been used for OTP-related vulnerability exploration. Its limitation lies in cases where apps implement SSL Pinning, a mechanism that employs a fixed SSL certificate verification method with the cloud, rendering man-in-the-middle attacks ineffective. Additionally, when there are encryption transformations and integrity verification mechanisms in the request packets, parameter mutation becomes impractical.

SACIntruder [12] focuses on the exploration of OTP Brute-Force vulnerabilities related to password reset. It achieves this by initiating each activity of the application to identify the password reset user interface (UI). Once the password reset UI is identified, it requests a SMS code with the victim's account and then attempts to brute-force the SMS code using the UI. The limitation of SACIntruder is that it specifically targets OTP vulnerabilities related to password reset, and its mutation occurs on the UI, making it unable to bypass the application's input filtering. Consequently, this framework cannot be utilized for detecting other OTP vulnerabilities.

AUTH-EYE [13] introduces a set of rules for OTP. It is a system that includes a login code detector and an auth message analyzer. It detects potential rule violations by analyzing network (and SMS) requests and responses. However, its drawback lies in its focus on checking whether the OTP implementation violates rules, without engaging in true OTP vulnerability exploration. Moreover, its detection method relies on Burp Suite and is still constrained by the SSL Pinning mechanism.

AuthChecker [14] is a semi-automated detection tool designed for SMS OTP brute-force cracking vulnerabilities. It utilizes an ART virtual machine plugin to capture runtime information in real-time, effectively bypassing protection mechanisms deployed on the application side or UI layer. However, when we extended the tool's application scope to other types of OTP vulnerabilities (such as OTP Bomb Vulnerability), we found that its existing tampering point screening mechanism has certain

then mutates parameters of the identified *cloud triggers* while monitoring server responses and callback function states. Finally, the framework outputs potential vulnerabilities detected through this process.

Static Analysis Module: The input is the Android Application Package (APK). The module first extracts the function call graph from the decompilation code. Then it identifies functions that communicate with the cloud, typically web functions or Restful APIs. The output is the *hook chain* of function calls that may represent the app's communication with the cloud.

Dynamic Analysis Module: This module uses the static analysis of *hook chains* as input. Firstly, it initiates the app and replays pre-recorded UI behaviors. Then it tracks the parameter and data flow changes of each hook function. This process helps us identify the last unchanged function of parameters before encryption and the last unchanged function of parameters before integrity checking, defined as *cloud triggers*. All *cloud triggers* are then output as variant points for runtime fuzzing.

Fuzz Testing Module: According to the dynamic and static analysis has been precisely located to the best point of mutation, where the mutation can not only bypass the previous format checking function, but also automatically generate a request packet in line with the encryption protocol. Continue to replay the recorded UI behavior, and mutate key parameters (such as authentication codes, phone numbers, etc.) during the runtime. For apps that can perform man-in-the-middle attacks a response-based state monitoring method is used, which determines whether the mutation is successful by comparing the similarity of the responses. For apps with SSL Pinning or non-application layer protocols enabled (i.e., not capable of man-in-the-middle attacks), the success of the mutation is monitored through callback functions (e.g., onSuccess, onFailed) in the hook code. This module takes *cloud triggers* as input. Mutating the parameters of *cloud triggers* allows bypassing the preceding format checking function and automatically generating a request packet compliant with the encryption protocol. It then continues to replay the recorded UI behavior and mutate key parameters (such as verification codes, phone numbers, etc.) during runtime. For apps capable of man-in-the-middle attacks, we employ a response-based state monitoring method. This method determines the success of the mutation by comparing the similarity of responses. For apps with SSL Pinning or non-application layer protocols enabled (i.e., incapable of man-in-the-middle attacks), we employ a callback function based state monitoring method. This method determines the success of the mutation by monitoring through callback functions' state (e.g., onSuccess, onFailed) in the hook code.

Manual Work, through our approach, testers only need to record a UI behavior in advance. This approach eliminates the need for testers to invest significant time in reading and understanding decompiled obfuscated code. It significantly enhances the ability to detect OTP-related vulnerabilities in the cloud.

B. Static Analysis

The static analysis module takes APK as input (all of this paper uses the app package of Android platform as the research

object). The module's aims to locate the chain of function calls related to the cloud communication.

Soot [16] is currently the most mainstream Java static analysis tool. We use pysoot [17] under the framework of angr [18] to analyze the APK. Pysoot is a Python version of a static analysis tool similar to Soot. Its kernel also utilizes pre-compiled Soot Jar packages for the analysis of Java code work. The steps of static analysis can be divided into three steps: call graph extraction, network communication function extraction, and hook chains construction.

1) Call Graph Extraction: The static analysis module firstly needs to extract the function call graph in the code. However, during the construction of the call graph by Pysoot, challenges arise due to newly prevalent Android development design patterns. For instance, the widely used network request framework, Retrofit2 [9] + RxJava2 [19] + OkHttp3 [20], adopts asynchronous multithreading and involves a substantial amount of function rewriting. For instance, when we use pysoot for static analysis, we find that the call graph is often incomplete due to implicit calls to asynchronous functions. In addition, Java development nowadays contains a lot of function rewriting. When the base class pointer is pointing to the derived class, pysoot can not correctly recognize the function call, resulting in the destruction of the call graph. These pose significant challenges for static analysis.

To solve the above challenges, we built separate rules for RxJava2 and Runnable interfaces:

- If *class.interfaces* is one of *ObservableOnSubscribe*, *SingleOnSubscribe*, *CompletableOnSubscribe*, *FlowableOnSubscribe*, *MaybeOnSubscribe*, add an edge from <init> to the <subscribe> function.
- If *class.interfaces* is *java.lang.Runnable*, add an edge of <init> to <run> functions.

For example, We analyze the AiHome application implemented with RxJava2 framework (as shown in Fig. 5). The decompiled code fragments are processed using Jadx [21] tool, with logs and irrelevant code removed for improved clarity. The implementation handles verification code validation for password reset through four key components: ForgetPasswordActivity (a) collects and validates user credentials before invoking Service; Service (b) wraps AccountInterface and returns an Observable; AccountObservable (c) implements subscribe to trigger verifySmsCode; and WebApi (d) executes the final POST request. The call graph fails to connect (a) to (d) because RxJava2's chained Observable mechanism implicitly invokes AccountObservable.subscribe() through subscribeActual rather than explicit calls. This dynamic dispatch behavior (mediated through base classes) prevents static analysis from resolving the concrete implementation, breaking the (a) to (c) call graph connection. Since in the vast majority of cases, the <subscribe> function is called in a singleton pattern, the <init> and <subscribe> methods always appear in pairs. Therefore, our rule ensures that the <init> constructor and the <subscribe> function of the ObservableOnSubscribe class are linked, maintaining the call graph's completeness.

2) Network Communication Function Extraction: After generating a complete call graph of the APK, we proceed to extract

```

1 public class ForgetPasswordActivity {
2     public final void forget() {
3         String account = mEditAccount.getText().toString();
4         String code = mEditVerification.getText().toString();
5         if (account.length() > 0 && code.length() > 0) {
6             Service.forget(account, code).subscribe(new
              Observer(account));
7         }
8     }
9 }
10 (a) UI Activity: Account and Code Verification with Format

11 public class Service implements AccountInterface {
12     public Observable<Unit> forget(String str, String str2) {
13         return AccountInterface.forget(this, str, str2);
14     }
15 }
16 public interface AccountInterface {
17     public static Observable<Unit> forget(AccountInterface
18     interface, String str, String str2) {
19         return Observable.create(new AccountObservable(
20         interface, str, str2));
21     }
22 }
23 (b) Service Interface: Wrapping AccountInterface for Observable

24 public class accountobservable <T> implements
25     ObservableOnSubscribe<Unit> {
26     AccountObservable(AccountInterface interface, String str,
27     String str2) {
28         this.interface = interface;
29         this.account = str; this.code = str2;
30     }
31     @Override
32     public final void subscribe(ObserverEmitter<Unit> e) {
33         Saas_accountKt.verifySmsCode(this.account, this.code);
34         e.onNext();
35     }
36 }
37 (c) Observable Logic: Subscription and Network Request Trigger

38 public static Observable<GHToken> verifySmsCode(String str,
39     String str2) {
40     return WebApi.sendRequest(CLOUD_URL, Enum.POST, "/users/
41     smsCode/verify", encode(str, str2).toString());
42 }
43 (d) Network Request: WebAPI POST for smsCode Validation

```

Fig. 5. Simplified Decompiled Code of AiHome's Password Reset Flow via Jadx (the same color represents parameter flow connections in the call graph).

the functions of interest that communicate with the cloud. An app's communication with the cloud usually ends with a call to a network-related function at the end of the call graph. So we need to locate all the network-related functions and search up the chain of all the calls that can get to the point of this function.

When a function call starts with the specified package name, it is identified as a network communication function. The package names are as follows:

- com.loopj.android.http, android.webkit, org.apache.http, java.net, com.android.volley, javax.net, android.net, okhttp3

Extracting network communication functions is essentially looking for the end point of the call graph. However, when the Android application adopts the Retrofit2 framework, the network communication functions are transformed into RESTful interfaces. Retrofit2 encapsulates the http network request framework, and the underlying communication is still done by okhttp. Since Retrofit2 ends with a RESTful interface, it is not possible to use the previous extraction rules. Therefore, we formulate specific extraction rules for communication functions for the characteristics of Retrofit2 to improve the static analysis.

Retrofit2 represents http requests as annotations, and constructs request packages by passing in parameters with fields such as Query, Header, and so on. According to this feature, we directly utilize the anti-obfuscation capability of Jadx tool, Jadx can reduce the obfuscated Java annotations from @a to @POST, @GET. So we can directly get all the RESTful interface functions according to the regular matching (matching all the Retrofit2 annotated keywords), as the end of the call graph.

In situations where the Native layer is invoked to transmit packets, it is challenging to ascertain the specific purpose of the

Algorithm 1: BFS Algorithm for Extracting Hook Chains.

Input: *net_functions*, *call_graph*

Output: *to_hook_functions*

```

1: function BFS()
2:   depth = 10
3:   while net_functions.size() > 0 and depth > 0
4:     func ← net_functions.front()
5:     to_hook_functions.push(func)
6:     net_functions += call_graph.get_caller(func)
7:     depth ← depth - 1
8:   return to_hook_functions

```

native interface. Therefore, we treat all Native interface functions as network communication functions. Dynamic analysis is then tasked with determining whether there is a data flow through the functions in the native layer.

3) *Hook Chains Construction*: To minimize the overhead of dynamic analysis, we examine all the call graph and identify that function chains associated with network communication typically consist of fewer than 10 functions. Consequently, we employ the breadth-first search algorithm to explore call graphs that can reach network communication functions limiting the search depth to 10. In addition, we also carry out function de-duplication, so that the number of the extracted hook chains are greatly reduced. Thus it can reduce the overhead of dynamic analysis of hooks. BFS Algorithm is shown in Algorithm 1. The input is the set of network communication functions and call graph, and the output is the set of functions that need to be hooked during dynamic analysis. Each function to be hooked is identified by *[class_name, method_name, [param1, ...], ret_type]*. The algorithm first initializes depth (Line 2), when the network function and depth are greater than 0 (Line 3), the top network function is added to the function that needs to be hooked (Lines 4-6), and finally depth is updated (Line 7).

C. Dynamic Analysis

Upon completion of static analysis, the system generates a list of all the functions that require hooking. This list encompasses all functions essential for the communication process between the app and the cloud. Dynamic analysis aims to locate the most suitable variant from hundreds of hook functions. In order to overcome **Challenge (1)**, we need to vary the parameters before the encryption function and before the integrity checking function. We utilize the encryption code already present in the app, allowing the automatic construction of request packets that adhere to the communication protocol. For **Challenge (3)**, there are usually some format checking and special character filtering functions in the app. Mutating after the format checking function can also improve the success rate of the fuzz testing and find more vulnerabilities. We define the mutation point before encryption function and after format checking as *Cloud Trigger*. Dynamic analysis identifies the *Cloud Trigger* through UI behavior replay and data flow tracking. This trigger guides the subsequent mutation actions in fuzz testing.

The UI behavior replay uses the record replay function of NoxPlayer [22]. Since the vulnerability we are concerned about is the OTP-related vulnerabilities, i.e., it is only related to logging in, registering, and password reset. So in the testing process, we pre-record the UI behaviors triggering the request for OTP and submission of OTP. Then we replay the recorded behaviors during dynamic analysis and fuzz testing.

We utilize Frida [23] for runtime hooking. For data flow tracking, during the recording of UI behavior, we input a predefined set of data, including a fixed phone number, email address, etc. These data are initially transmitted in plaintext within the app, such as `stringuser = this.editText.getText().toString()`. We leverage this capability to monitor data flow. If the parameter of a function call lacks plaintext data, it indicates that the parameter has undergone cryptographic transformation. In such cases, *Cloud Trigger* is identified as the last function where plaintext data appears as a parameter.

For the case that apps often add sign as an integrity check in the request parameters, we mark each function for the presence of md5, sha1, base64, hmac (common signature functions). When the hook function contains a signature algorithm, we consider *Cloud Trigger* to be the last triggered hook function. The specific extraction method is shown in Algorithm 2, where the input is the result of static analysis and the output is *Cloud Trigger*. Firstly, based on the Frida tool in the JavaScript layer, we hook all the *to_hook_functions* (Line 2), and then we replay the UI behaviors (Line 3), and when triggered to a hook function, we call the callback function to notify the python layer of the current invoke function (Line 6), and determine if the function's arguments contain the plaintext value of phone number, authentication code, or email address, then the invoke function is considered as a potential trigger (Lines 8-9). When the function contains hash functions such as md5, sha-1, sha-2, base64encode, hmac, etc., or when the argument does not contain a plaintext value (Lines 10-13), the previous potential trigger is output as the best variant.

D. Fuzz Testing

In fuzz testing, we mutate the input data thereby mining the security boundaries of the target cloud. Fuzz testing usually consists of three parts: seed input, mutation rules and state monitoring. In the scenario of mining OTP-related vulnerabilities, we have identified the mutation point *cloud triggers*. The input seeds are the parameters of the cloud triggers, such as phone number, verification code, email, etc. Next we will introduce the mutation rules, status monitoring and vulnerability determination rules.

1) *Mutation Rules*: Since we are targeting OTP-related vulnerabilities, certain rules need to be met when mutating. Therefore the mutation rules during fuzzing are formulated according to each of the four types of vulnerabilities mentioned before:

- *OTP Leakage Vulnerability*: No mutation is required and the behavior of requesting OTP is replayed directly 5 times.
- *OTP Brute-Force Vulnerability*: The modified verification code field adheres to the original code's length, randomly generating digits ranging from 4 to 6, and repeating this process 30 times.

Algorithm 2: Algorithm for Extracting Cloud Triggers.

Input: *to_hook_functions*

Output: *cloud_trigger*

```

1: function Get_Cloud_Trigger()
2:   // hook all functions in [to_hook_functions] by Frida
3:   // replay UI activity
4:   phone, code, email = 137xxx, 1234, abc@gmail.com
5:   cloud_trigger ← ∅
6:   function onHookCallBack()
7:     invoked_func = *this
8:     if invoked_func.contains_arg
        (< phone, code, email >)
9:       last_func ← invoked_func
10:    else if invoked_func.contains_func
        (< md5, sha1, sha2, base64, hmac >)
11:      cloud_trigger ← last_func
12:    else // not contain a plaintext value
13:      cloud_trigger ← last_func
14:    return cloud_trigger

```

- *OTP Bomb Vulnerability*: For the case of SMS verification code, we directly replay it 30 times, then add special characters such as '\t', '\n', '+', '\x00' at the beginning and end of the phone number field. For the case of email verification code, we first replay it 30 times and then mutate it (e.g., +86) for the area code field. Because the area code has no significance for mailboxes, the area code and mailboxes are often checked together in duplicate checking in many clouds.
- *Resource Consumption Vulnerability*: For the case of SMS verification code, the phone number is mutated to a total of 50 times to the phone number provided by the network code receiving platform. For the case of email verification code, the email address is mutated to a randomly generated 50 numbers, which contain several valid addresses for confirming whether the sending behavior is successful or not.

When designing OTP verification systems, vendors may impose limitations on the frequency of requests within a specified time frame. This is done to prevent the misuse of verification codes. Considering this, we opt for a higher number of mutation instances to minimize the false positive rate associated with potential vulnerabilities.

2) *Status Monitoring*: In the status monitoring, we are unable to observe the operation status of the cloud, so we can only determine whether the mutation is successful or not based on the return information from the cloud.

For apps that can perform man-in-the-middle attacks and can intercept http and https traffic, we use a state monitoring method based on response message comparison. The first correct response is used as a benchmark, and the success of the mutation is judged based on the similarity between the response obtained from the mutation request and the benchmark. Traffic proxy capture is done using Burp Suite [11] and Charles [24] tools. Since cloud responses often contain values such as timestamps,

temporary tokens, and other values that change each time, it is not possible to make a judgment based solely on whether the responses are the same. However, it is necessary to make a comparative judgment based on the meaning of each field in the response.

We have designed the status monitoring algorithm based on response messages comparison, as shown in Algorithm 3. The inputs are the correct response benchmark and the mutated response awaiting judgment, and the output determines whether the mutation is successful or not. For the judgment of 80% in the response, we consider that the results returned exhibit a certain degree of inconsistency. We found that choosing 80% effectively reduces the false-negative rate.

- 1) If the status code of the two responses is different, the mutation fails (Lines 2-3);
- 2) If the response content is completely consistent, the mutation succeeds (Lines 4-5);
- 3) If in the response there is a key-value pair with the value of [0,1,True,False], it is very likely that it is a field identifying the result of the response. It is judged according to the value when 80% of the fields identifying the response result in both responses are the same. Then the mutation is judged to be successful (Lines 6-10);
- 4) If there is a key-value pair with key named ["statusCode", "result", "code", "state", "status", "status_code"] in the response, then its most likely to be the field identifying the response result. And it is judged according to value, when 80% of the fields identifying the response result are the same in both responses. Then the mutation is determined to be successful (Lines 11-16).

For apps that are unable to capture traffic, i.e., with SSL Pinning or transport layer protocols enabled, we adopt a callback function-based state monitoring approach. The inspiration for this approach comes from the observation that when the cloud returns a response result, the app's UI interface typically displays a corresponding prompt. Consequently, different response results, i.e., callback functions, are processed in the app code. For example, in the RxJava2 framework, *Observer* has different callback functions to handle the signals from *Observable*. Therefore, for apps that cannot capture packets, we choose *onSuccess*, *onComplete* functions as the status monitor for success and *onError* as the status monitor for failure.

3) *Vulnerability Determination Rules*: Since we focus on 4 types of OTP-related vulnerabilities, we have different success determination criteria from traditional fuzzing. Traditional fuzzing determines that a variant is successful when the state monitoring reaches the target running down or downtime. Our state-based monitoring approach can only know whether the cloud is responding correctly or incorrectly, but cannot determine the true operational state of the cloud. Therefore we also need to design vulnerability determination rules for these 4 types of OTP-related vulnerabilities:

- *OTP Leakage Vulnerability*: If the 4-6 digit alphanumeric combination consistently appears in the cloud response or callback function variable each time, it indicates the existence of an OTP leakage vulnerability.

Algorithm 3: Status Monitoring Algorithm Based on Response Messages.

Input: *success_resp*, *fuzzing_resp*

Output: *fuzzing_result*

```

1: function Is_Fuzzing_Success()
2:   if success_resp.status_code ≠ fuzzing_resp.status_code
3:     return False
4:   if success_resp.text == fuzzing_resp.text
5:     return True
6:   flag_keys ← ∅
7:   for key, value in success_resp.json()
8:     if value ∈ [0, 1, True, False]
9:       flag_keys.append(key)
10:    if percent_of(success_resp.get(flag_keys) == fuzzing_resp.get(flag_keys)) > 0.8
11:      flag_keys_2 ← ∅
12:      for key, value in success_resp.json()
13:        if key ∈ ["statusCode", "result", "code", "state", "status", "status_code"]
14:          flag_keys_2.append(key)
15:        if percent_of(success_resp.get(flag_keys_2) == fuzzing_resp.get(flag_keys_2)) > 0.8
16:          return True
17:    return False

```

- *OTP Brute-Force Vulnerability*: If the cloud response is identical in all 30 variants, it is determined that an OTP burstable vulnerability exists.
- *OTP Bomb Vulnerability*: If the number of replay successes exceeds 20, it is determined that an OTP bomb vulnerability exists.
- *Resource Consumption Vulnerability*: If the replay succeeds more than 45 times out of 50 (excluding misses due to network reasons), it is determined that a resource consumption vulnerability exists.

IV. EXPERIMENTAL EVALUATION

In this section, we answer three research questions to evaluate the effectiveness of the proposed framework:

- **RQ1**: Is the proposed framework effective in detecting OTP-related vulnerabilities?
- **RQ2**: Are the static and dynamic analysis modules of the framework effective?
- **RQ3**: Compared with previous research and testing methods, what are the advantages of the framework?

To answer the first research question, we have evaluated OTP-Hunter's ability in detecting OTP-related vulnerabilities (Section IV-B). Our system found 68 OTP-related vulnerabilities in 59 apps.

To answer the second research question, we have analyzed the results of static and dynamic analysis. From the results, we can see that the difficulty and workload of program analysis is greatly reduced by these two analyses. In addition, we illustrate

through ablation experiments that the special rules we added in the static analysis are effective for extracting the call graph (Section IV-C).

To answer the third research question, we have compare it with traditional testing methods and related work on OTP vulnerability mining. From the results, we see that our system overcomes the problems of existing methods in detecting OTP-related vulnerabilities, and has a wider detection scope (Section IV-D).

A. Environment Setup

In order to verify the practical effect of fuzz testing framework, we have implemented the prototype system OTP-Hunter. OTP-Hunter utilizes the pysoot framework for static analysis, involving approximately 3,212 lines of Python code. Additionally, dynamic analysis and fuzz testing are implemented using the frida framework, with a combined codebase of around 1,036 lines encompassing both JavaScript and Python. The test environment is Night God Android Emulator version 7.0.2.6 32-bit, with performance settings of 4-core CPU, 4 G RAM, and Android version 7.1.2. The emulator run on a computer (Intel Core i7-6700HQ 2.60 GHz CPU, 16 G RAM) running Windows 10.

We collected 59 IoT-related apps for testing, and the download sources of APK are mainly Google Play [25], apkpure [26] and vendor's official website. We have found a total of 68 OTP-related vulnerabilities. Due to ethical considerations and legal restrictions, the test targets were all cloud accounts applied by ourselves, and did not affect other people's cloud accounts. The discovered vulnerabilities have been reported to vendors and China National Vulnerability Database (CNVD), and have not been exploited for further attacks.

B. Fuzzing Result

Based on the results of the dynamic analysis, we have conducted fuzz testing on 43 apps that successfully extracted *Cloud Trigger* among 59 tested apps. In total, OTP-Hunter have discovered 68 OTP-related vulnerabilities. Our vulnerability detection results answer **RQ1** well, OTP-Hunter is effective in detecting the IoT cloud account management vulnerabilities.

The number of various types of OTP-related vulnerabilities detected by OTP-Hunter is shown in Fig. 6. CNVD scores vulnerabilities based on their level of impact and exploitability. OTP leakage vulnerability belongs to high-risk vulnerability, OTP brute-force vulnerability belongs to medium-risk vulnerability, and OTP bomb and resource consumption can be regarded as low-risk vulnerability. We chose IoT-related apps because of the amplified hazards of OTP-related vulnerabilities in IoT. OTP leakage vulnerabilities and OTP brute-force vulnerabilities can allow attackers to take over the other's cloud account and remotely view and control other people's smart homes by simply knowing the other's account. There are a total of 18 OTP leakage and OTP brute-force vulnerabilities, which account for 26.4% of all the vulnerabilities. OTP bomb and resource consumption vulnerabilities can lead to the consumption of vendor's resources, which is a low-risk vulnerability. There are a total of 50 OTP

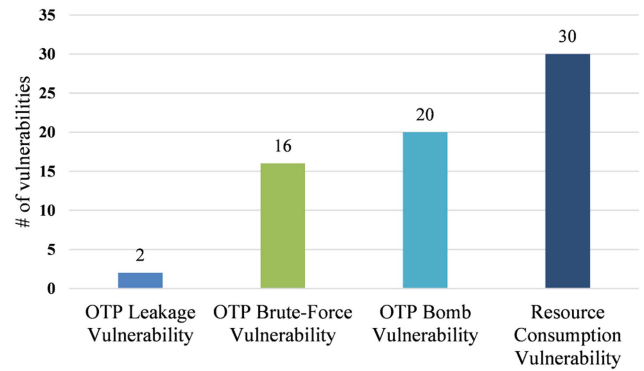


Fig. 6. Results of OTP-related vulnerabilities detected by OTP-Hunter.

Bomb and resource consumption vulnerabilities, accounting for 73.6% of all vulnerabilities.

The OTP vulnerabilities found in the experiments have been reported to the vendors or China National Vulnerability Database (CNVD), and some of the vulnerabilities authorized for public disclosure are shown in Table I.

C. Static Analysis and Dynamic Analysis Result

To answer **RQ2**, we conducted experiments on static and dynamic analysis. In static analysis, we showcase our capability to extract hook chains and emphasize the significance of incorporating our specialized rules through ablation experiment. Additionally, we demonstrate that static analysis effectively reduces the workload associated with dynamic analysis. In dynamic analysis, we use OTP-Hunter to extract *Cloud Trigger* (which is the key to perform fuzz testing), illustrating that dynamic analysis facilitates fuzz testing.

1) *Static Analysis Result*: For the 59 apps to be tested, we use the static analysis method for hook chain extraction to evaluate the effect of static analysis, and the experimental results are shown in Fig. 7(a). Among the 59 apps under testing, the static analysis module has successfully extracted the hook chain of 49 apps, which can cover the function call chain during the communication process between the app and cloud. In the process of static analysis, 50 apps can successfully extract Call Graph, and 49 apps can successfully extract network communication functions. Among the 10 apps that failed to extract, 9 were shelled apps, and the static analysis tool could not automatically reverse the code. And 1 was an app with an embedded web page, which could not locate the related network communication functions.

Ablation experiment: In addition, we have also conducted ablation experiment on the effect of call graph extraction for static analysis, as shown in Fig. 7(b). From the results, it can be seen that if the special rules for apps are not enabled during static analysis, 11 apps will have missing edges during call graph extraction, resulting in broken function call chains. The experiments show that the special rules for RxJava2, Runnable are effective.

In dynamic analysis, hooking all functions in an APK (approximately 110,000 on average) caused significant

TABLE I
CNVD NUMBERS OF SOME OTP VULNERABILITIES OF IOT CLOUD (RATINGS IS ASSIGNED BY CNVD BASED ON VULNERABILITY IMPACT AND EXPLOITABILITY LEVEL)

IoT App Package Name	CNVD number	Vulnerability Type	Vulnerability Ratings
com.starnet.aihomeintl.pro	CNVD-2022-77920	Arbitrary Password Reset	Medium
com.kiwik.usmartgo	CNVD-2022-77922	Arbitrary Password Reset	Medium
com.yinkou.YKTCProject	CNVD-2022-77939	Arbitrary Password Reset	High
com.thinkhome.v3	CNVD-2022-77940	Arbitrary Password Reset	Medium
com.kuju.smarthome	CNVD-2022-78466	Arbitrary Password Reset	Medium
com.gehua.smarthomemobile	CNVD-2022-78468	Arbitrary Password Reset	Medium
com.ys7.enterprise	CNVD-2022-89217	SMS Bomb	Medium
com.uniation.kyy	CNVD-2022-89218	Arbitrary Password Reset	Medium
com.tange365.icam365	CNVD-2022-89576	Arbitrary Password Reset	Medium
com.jooan.qalink	CNVD-2022-89577	Arbitrary Password Reset	Medium
com.generalcomp.luowice	CNVD-2022-89598	Arbitrary Password Reset	Medium

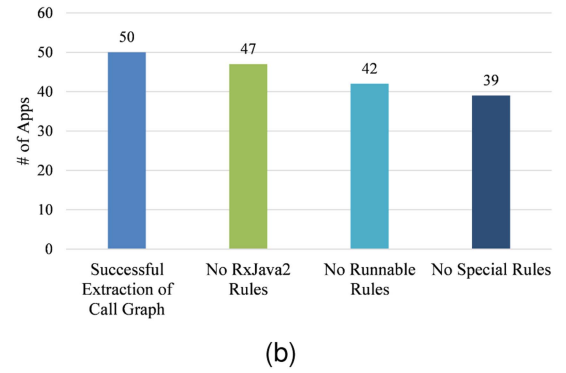
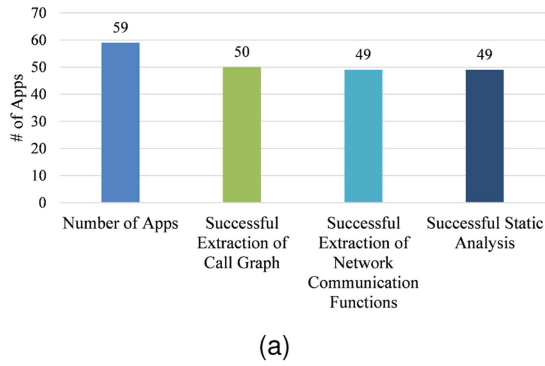


Fig. 7. Experiment results of static analysis (a) Number of successful apps in static analysis. (b) Call graph ablation experiment.

TABLE II
EXPERIMENT RESULTS OF DYNAMIC ANALYSIS

	Number of Apps
Total	59
Extracted Hook Chain in Static Analysis	49
Extracted Cloud Trigger in Dynamic Analysis	43

performance overhead and often triggered crashes due to bad classes. To efficiently and stably capture cloud communication behaviors, we employed static analysis to precisely identify critical functions. Experiments demonstrated that hooking only around 1,986 functions was sufficient to cover all network-related calls, significantly improving analysis efficiency and reducing crash risks. This confirmed the necessity and effectiveness of static analysis in optimizing dynamic hooking.

2) *Dynamic Analysis Result*: The results of the dynamic analysis experiments are shown in Table II. In the experiments of dynamic analysis, we regard the functions that can be captured with parameters such as phone number and authentication code as successfully extracted *Cloud Trigger*. Among the 59 tested apps, 43 of them are able to locate *Cloud Trigger* successfully. The extraction failures are shown in Fig. 8. Among them, 9 of them are because the APK is shelled, which makes it difficult to perform both static and dynamic analysis. 4 of them are due to the fact that the APK directly crashes after hook, which makes it impossible to perform subsequent tests. 3 of them are due to

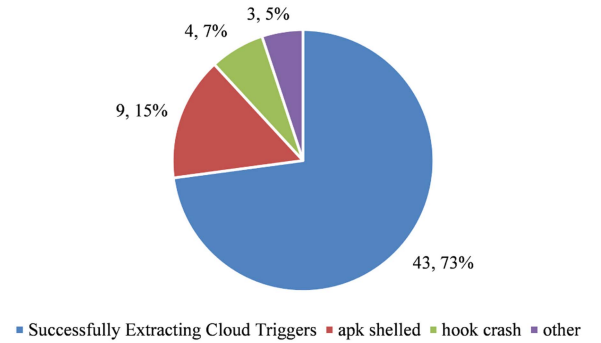


Fig. 8. Reasons for failure in dynamic analysis.

other different reasons, such as the login and registration page of a certain vendor is an embedded Web page, which makes it impossible to obtain the direction of the input data flow from the inside of the app.

It can be seen that, except for 9 shelled apks, the static and dynamic analysis modules can successfully analyze most of the apks. They extract the necessary *cloud triggers* for fuzz testing and identify the optimal mutation location for subsequent testing.

D. Comparison

To answer **RQ3**, we first analyzed the number of vendors with OTP vulnerabilities where app protection exists. The main

TABLE III
COMPARISON WITH OTHER TOOLS

Tool	Detection Scope	Human Involvement	Data Mutation Method	Bypass Anti-tampering
SACIntruder [12]	OTP Brute-Force Vuln	Less	UI Mutation	✓
AUTH-EYE [13]	OTP Brute-Force Vuln	More	No Mutation	×
AuthChecker [14]	OTP Brute-Force Vuln	Less	Runtime Mutation(Coarse)	✓
Burp Suite [11]	More OTP Vulns	More	HTTP Mutation	×
OTP-Hunter	More OTP Vulns	Less	Runtime Mutation(Precise)	✓

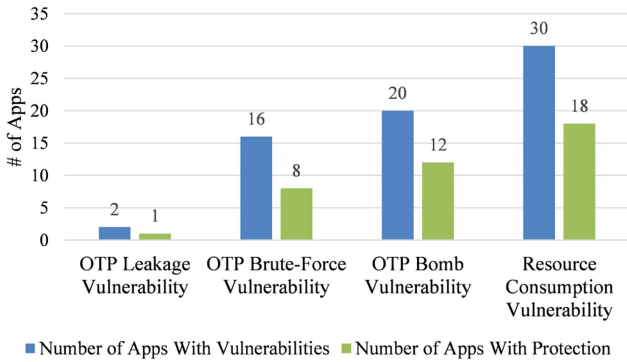


Fig. 9. Applications with OTP vulnerabilities and the presence of protection in application (among the 68 apps containing vulnerabilities, 39 apps have this app protection).

protection measures include: parameter encryption, integrity checking, SSL Pinning and so on. The apps with protection cannot be directly exploited by traditional logical vulnerability mining methods(e.g. Burp Suite [11], Charles [24]) such as packet capture, proxy traffic, and mutation replay. The usual manual practice is to bypass the protection by reversing or replacing certificates and other operations before replay. However, OTP-Hunter can save a lot of reverse engineering. The analysis results, depicted in Fig. 9, reveal that 57% of the OTP-related vulnerabilities discovered by OTP-Hunter are present in protected apps. This finding highlights the potential for a high under-reporting rate when employing traditional methods, a gap effectively addressed by our proposed OTP-related vulnerability fuzz testing framework.

OTP-Hunter specializes in identifying vulnerabilities related to OTPs, and we benchmark its findings against those of other relevant research endeavors. SACIntruder [12] is a tool for automatically detecting SMS code brute-force attack. AUTH-EYE [13] is a system for automated detection of OTP rule violations. AuthChecker [14] is a semi-automated tool for detecting SMS OTP brute-force vulnerabilities. Burp Suite [11] is a traditional Web OTP vulnerability testing tool. Since the source code of SACIntruder, AUTH-EYE and AuthChecker are not publicly available, it is not possible to compare and test the 59 tested apps in the experiment. In this paper, we have analyzed the testing principles and compared the advantages and disadvantages of the currently existing research as well as the traditional web OTP vulnerability testing with our proposed fuzz testing framework. Our evaluation metrics include: detection scope, human involvement, data mutation method, and bypassing anti-tampering. The comparison results are shown in Table III.

(1) *Detection Scope*: The evaluation reveals OTP-Hunter's superior vulnerability coverage compared to other tools. Unlike SACIntruder, AUTH-EYE and AuthChecker that only detect brute-force attacks, OTP-Hunter successfully identifies various OTP-related vulnerabilities including brute-force attacks, bombing vulnerabilities, and implementation flaws. While Burp Suite offers general testing functionality, it lacks OTP-specific optimizations found in OTP-Hunter. In terms of required human involvement, Burp Suite demands security experts with deep OTP vulnerability knowledge for effective testing, and AUTH-EYE requires additional manual intervention due to its unreliable login activity detection, whereas OTP-Hunter achieves automation with minimal human input.

(2) *Human Involvement*: The effectiveness of Burp Suite heavily relies on testers with specialized expertise in OTP vulnerability detection. In contrast, OTP-Hunter achieves comparable results with significantly reduced dependence on human expertise, requiring only the recording of UI behaviors to enable automated vulnerability discovery.

(3) *Data Mutation Method*: The tools employ fundamentally different approaches to data mutation with varying effectiveness. AUTH-EYE provides no mutation capability during testing, severely limiting its detection scope. SACIntruder's UI-level mutations prove inadequate as they cannot bypass application input validation. AuthChecker employs an overly broad mutation strategy that targets all Java functions handling OTP parameters or return values. This approach necessitates extensive hooking operations and frequently proves ineffective, as the mutations are applied prior to input validation checks. In contrast, OTP-Hunter's methodology combines static and dynamic analysis to precisely identify optimal *cloud triggers* for mutation, then employs runtime mutation to generate data that successfully bypasses format checks. This proves particularly effective against OTP bombing vulnerabilities - when testing phone number validation, OTP-Hunter's mutated inputs containing special characters (like `\t`, `\n`, `\x00`) exploit timing gaps between validation and filtering checks. The cloud service interprets these specially crafted numbers as unique entries due to the pre-filter validation, enabling successful bombing attack simulation where other tools fail. This sophisticated mutation approach demonstrates OTP-Hunter's technical superiority in circumventing complex input validation scenarios.

(4) *Bypass Anti-tampering*: OTP-Hunter employs innovative code reuse techniques to uniquely overcome application protection mechanisms. This approach successfully bypasses critical security measures including SSL Pinning, parameter encryption, and integrity checks - limitations that significantly constrain the effectiveness of both AUTH-EYE and Burp Suite in real-world testing scenarios.

V. DISCUSSIONS AND LIMITATIONS

To mitigate OTP vulnerabilities, we recommend implementing the following critical measures: First, all OTP verification must be completed on the cloud side, strictly prohibiting the delegation of verification logic to client-side implementations. Second, enforce rigorous access control policies, including limitations on OTP attempt counts and request frequency thresholds. Simultaneously, ensure encrypted transmission of OTPs while implementing real-time monitoring for anomalous request behaviors. From an architectural design perspective, developers must adhere to the "zero trust" principle, never implicitly trusting any client-side requests by default. Cloud providers should implement robust rate-limiting and threat intelligence sharing to protect against OTP vulnerabilities at the infrastructure level.

Our proposed OTP-related vulnerability fuzzing testing framework can be well applied to the detection of all OTP vulnerabilities. This approach can bypass the four common means of protection on the app side. However, this framework is based on the results of static and dynamic analysis, and cannot be applied to shelled APK. One of the major reasons for the failure of extracting *Cloud Trigger* during the experiments is that the APK is protected by shelling. Even though the tool based on frida-dexdump [27] can be dynamically loaded to remove the shell, it is still difficult to realize the penetrating hook function. Therefore, for the shelled APK, it is difficult to use OTP-Hunter for vulnerability detection. In addition, OTP-Hunter currently lacks applicability for vulnerabilities arising from multi-step operations that result in unsynchronized states within the cloud. In our future work, we will work on improving the limitations of OTP-Hunter, solving the problem of APK shelling, and adapting to more complex vulnerability detection scenarios.

VI. RELATED WORK

OTP-based threat detection: Previous research focuses on analyzing user-side OTP leakage [28], [29], considering the possibility of OTP leakage from the generation of OTP [30]. Also some researchers focus on the way of using OTP [13], [31], [32], [33], and analyze the methods that may lead to OTP leakage [34], [35], [36], [37]. PRMitM [34] proposes a man-in-the-middle attack that exploits OTP during password reset, and finds that there is a flaw in the use of OTP on many websites. Bianchi et al. [28] develop an "identity-transfer" attack to locate insecure OTP authentication mechanisms. AUTH-EYE [13] defined six OTP rules based on RFCs, and infers the correctness of OTP by analysing server response. OTP-Lint [30] examines the OTP generation of apps to see if the app-generated OTPs are susceptible to attack. Lei et al. [35] systematically studied the attacker's method of obtaining OTPs by running an app on the victim's device, revealing the implementation errors of SMS OTP on mobile operating systems. AuthChecker [14] primarily targeted SMS OTP brute-force vulnerabilities. It employed a dynamic analysis method capable of bypassing applications with protection mechanisms at the application side or UI layer. Unlike previous studies, our work focuses on OTP vulnerabilities related to cloud account management in IoT scenario, which may lead to user information leakage, cloud resource consumption,

and other issues. We start from the communication side of the app and the cloud, and use a combination of static and dynamic analysis to detect these vulnerabilities.

Analysis Android apps: Some previous studies performed static [38], [39] and dynamic analysis [40], [41], [42] of mobile applications. JuCify [38] addresses the shortcomings of static analysis by extracting and merging call graphs of native code and bytecode to analyze where sensitive information goes in a program. T-Recs [40] addresses the current environment dependency of dynamic analysis by recording the bytecode level of an app on an Android device to track the information flow. OTP-Hunter is similar to DIANE [8] in that it finds triggers for the fuzz testing through a combination of static and dynamic analysis. But in terms of the scenarios targeted, DIANE mainly detects vulnerabilities in IoT devices, and our work mainly focuses on detecting OTP-related vulnerabilities. We have added some additional rules for the current mainstream framework (Retrofit2) to improve the success rate of recognizing triggers.

VII. CONCLUSION

We propose OTP-Hunter, a novel fuzzing framework that integrates static and dynamic analysis to automatically detect four types of OTP vulnerabilities in mobile applications. Our approach bypasses the limitations of requiring knowledge about encryption/SSL pinning, while incorporating specialized rules for Android frameworks to handle asynchronous calls and RESTful interfaces. By identifying *cloud triggers* through dynamic hooking techniques, we circumvent client-side input validation to conduct effective fuzzing. Evaluation on 59 IoT applications uncovered 68 real-world vulnerabilities, with 11 vulnerabilities assigned CNVD IDs, demonstrating our framework's practical effectiveness in enhancing mobile application security.

REFERENCES

- [1] M. Schneider, H. Shulman, A. Sidis, R. Sidis, and M. Waidner, "Diving into email bomb attack," in *Proc. 50th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2020, pp. 286–293.
- [2] TheSpeedX, "TBomb," GitHub, 2021. Accessed: Sep. 13, 2023. [Online]. Available: <https://github.com/TheSpeedX/TBomb>
- [3] Shellvon, "SMSBomb," GitHub, 2018. Accessed: Sep. 13, 2023. [Online]. Available: <https://github.com/shellvon/smsBomb>
- [4] Nikait, "Ni_bomber," GitHub, 2020. Accessed: Sep. 13, 2023. [Online]. Available: https://github.com/Nikait/ni_bomber
- [5] G. Nagli, "No rate limiting on https://[domain]/accounts/password/reset/endpoint leads to denial of service," HackerOne, 2020. Accessed: Sep. 13, 2023. [Online]. Available: <https://hackerone.com/reports/862681>
- [6] V. Devgan, "No rate limit on forgot password page of NordVPN," HackerOne, 2019. Accessed: Sep. 13, 2023. [Online]. Available: <https://hackerone.com/reports/751604>
- [7] Aliyugombe, "No rate limit in otp code sending," HackerOne, 2020. Accessed: Sep. 13, 2023. [Online]. Available: <https://hackerone.com/reports/1060518>
- [8] N. Redini et al., "Diane: Identifying fuzzing triggers in apps to generate under-constrained inputs for IoT devices," in *Proc. IEEE Symp. Secur. Privacy*, May 2021, pp. 484–500.
- [9] Square, "Retrofit2," GitHub, 2023. Accessed: Sep. 13, 2023. [Online]. Available: <https://square.github.io/retrofit>
- [10] Beijing Lenovo Software co., Ltd., "Lenovosmart apk," *Apkpure.com*, Accessed: Sep. 13, 2023. [Online]. Available: <https://apkpure.com/lenovosmart/com.lenovo.overseas>
- [11] PortSwigger Ltd, "Burp suite," Portswigger.net, 2023. Accessed: Sep. 13, 2023. [Online]. Available: <https://portswigger.net/burp>

- [12] D. Wang, X. Zhang, J. Ming, T. Chen, C. Wang, and W. Niu, "Resetting your password is vulnerable: A security study of common SMS-based authentication in IoT device," *Wireless Commun. Mobile Comput.*, vol. 2018, 2018, doi: [10.1155/2018/7849065](https://doi.org/10.1155/2018/7849065).
- [13] S. Ma et al., "An empirical study of SMS one-time password authentication in android apps," in *Proc. Annu. Comput. Secur. Appl. Conf.*, 2019, pp. 339–354.
- [14] J. Zhao, F. He, Y. Yang, and Y. Zhang, "Identifying implementation flaws of SMS OTP authentication," *IEEE Trans. Mobile Comput.*, early access, doi: [10.1109/TMC.2025.3550883](https://doi.org/10.1109/TMC.2025.3550883).
- [15] F. J. Ramírez-López, Á. J. Varela-Vaca, J. Roperio, J. Luque, and A. Carrasco, "A framework to secure the development and auditing of SSL pinning in mobile applications: The case of android devices," *Entropy*, vol. 21, no. 12, 2019, Art. no. 1136.
- [16] P. Lam, E. Bodden, O. Lhoták, and L. Hendren, "The soot framework for java program analysis: A retrospective," in *Proc. Cetus Users Compiler Infrastruct. Workshop*, 2011, pp. 1–8.
- [17] Angr, "Pysoot," GitHub, 2017. Accessed: Sep. 13, 2023. [Online]. Available: <https://github.com/angr/pysoot>
- [18] Y. Shoshitaishvili et al., "Sok:(state of) the art of war: Offensive techniques in binary analysis," in *Proc. IEEE Symp. Secur. Privacy*, 2016, pp. 138–157.
- [19] ReactiveX, "Rxjava2," GitHub, 2023. Accessed: Sep. 13, 2023. [Online]. Available: <https://github.com/ReactiveX/RxJava>
- [20] Square, "Okhttp3," GitHub, 2019. Accessed: Sep. 13, 2023. [Online]. Available: <https://square.github.io/okhttp>
- [21] Skylot, "JADX," Github, 2023. Accessed: Sep. 13, 2023. [Online]. Available: <https://github.com/skylot/jadx>
- [22] Yeshe, "Noxplayer," Yeshe.com, 2023. Accessed: Sep. 13, 2023. [Online]. Available: <https://www.yeshe.com/>
- [23] O. A. V. Ravns, "frida," Frida.re, 2023. Accessed: Sep. 13, 2023. [Online]. Available: <https://frida.re/>
- [24] XK72, "Charles," Charlesproxy.com, 2023. Accessed: Sep. 13, 2023. [Online]. Available: <https://www.charlesproxy.com/>
- [25] Google Inc., "Google play," Google, 2023. Accessed: Sep. 13, 2023. [Online]. Available: <https://play.google.com/store>
- [26] APKPure, "Apkpure," Apkpure.com, 2023. Accessed: Sep. 13, 2023. [Online]. Available: <https://apkpure.com>
- [27] Hluwa, "Frida-dexdump," GitHub, 2022. Accessed: Sep. 13, 2023. [Online]. Available: <https://github.com/hluwa/frida-dexdump>
- [28] A. Bianchi, E. Gustafson, Y. Fratantonio, C. Kruegel, and G. Vigna, "Exploitation and mitigation of authentication schemes based on device-public information," in *Proc. Annu. Comput. Secur. Appl. Conf.*, 2017, pp. 16–27.
- [29] A. Aparicio, M. M. Martínez-González, and V. Cardenoso-Payo, "App-based detection of vulnerable implementations of OTP SMS apis in the banking sector," *Wireless Netw.*, vol. 30, pp. 6451–6464, 2023.
- [30] S. Ma et al., "Fine with '1234'? An analysis of SMS one-time password randomness in android apps," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng.*, 2021, pp. 1671–1682.
- [31] C. Mulliner, R. Borgaonkar, P. Stewin, and J.-P. Seifert, "SMS-based one-time passwords: Attacks and defense: (short paper)," in *Proc. DIMVA*, Jul. 2013, pp. 150–159.
- [32] R. Kumar, S. Kishore, H. Lu, and A. Prakash, "Security analysis of unified payments interface and payment apps in India," in *Proc. USENIX Secur. Symp.*, 2020, pp. 1499–1516.
- [33] A. Gavazzi et al., "A study of multi-factor and risk-based authentication availability," in *Proc. USENIX Secur. Symp.*, 2023, pp. 2043–2060.
- [34] N. Gelernter, S. Kalma, B. Magnezi, and H. Porcilan, "The password reset MitM attack," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 251–267.
- [35] Z. Lei, Y. Nan, Y. Fratantonio, and A. Bianchi, "On the insecurity of SMS one-time password messages against local attackers in modern mobile devices," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2021.
- [36] W. Jin, X. Ji, R. He, Z. Zhuang, W. Xu, and Y. Tian, "Sms goes nuclear: Fortifying SMS-based MFA in online account ecosystem," in *Proc. 51st Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2021, pp. 7–14.
- [37] B. Tejaswi, M. Mannan, and A. Youssef, "Security weaknesses in IoT management platforms," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 1572–1588, Jan. 2024.
- [38] J. Samhi et al., "Jucify: A step towards android code unification for enhanced static analysis," in *Proc. IEEE/ACM Int. Conf. Softw. Eng.*, 2022, pp. 1232–1244.
- [39] T. T. Nguyen, M. Backes, N. Marnau, and B. Stock, "Share first, ask later (or never?) studying violations of {GDPR's} explicit consent in android apps," in *Proc. USENIX Secur. Symp.*, Aug. 2021, pp. 3667–3684.
- [40] H. Inayoshi, S. Kakei, and S. Saito, "Plug and analyze: Usable dynamic taint tracker for android apps," in *Proc. IEEE 22nd Int. Work. Conf. Source Code Anal. Manipulation*, 2022, pp. 24–34.
- [41] M. Diamantaris, E. P. Papadopoulos, E. P. Markatos, S. Ioannidis, and J. Polakis, "REAPER: Real-time app analysis for augmenting the android permission system," in *Proc. 9th ACM Conf. Data Appl. Secur. Privacy*, 2019, pp. 37–48.
- [42] C. Sun, Y. Ma, D. Zeng, G. Tan, S. Ma, and Y. Wu, "μdep: Mutation-based dependency generation for precise taint analysis on android native code," *IEEE Trans. Depend. Sec. Comput.*, vol. 20, no. 2, pp. 1461–1475, Mar./Apr. 2023.



Futai Zou (Senior Member, IEEE) received the PhD degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China, in 2005. He is associate professor with the School of Computer Science, Shanghai Jiao Tong University in China. His research interests include network attack and defense technology, software and system security. He is a senior member of CCF.



Zehui Zhang is currently working toward the master's degree with the School of Computer Science, Shanghai Jiao Tong University in China. His research interests include IoT security, fuzz testing, and penetration testing.



Yuzong Hu is currently working toward the master's degree with the School of Computer Science, Shanghai Jiao Tong University in China. His research interests include IoT security and fuzz testing.